

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026
umhackathon@um.edu.my

StockMaster AI

Team: Galaxy Annihilator

Table of Contents

1. Project Overview	1
2. Background & Business Objective	1
3. Product Purpose	2
4. System Functionalities	2
5. User Stories & Use Cases	3
6. Features Included (Scope Definition)	3
7. Features Not Included (Scope Control)	4
8. Assumptions & Constraints	4
9. Risks & Questions Throughout Development	5

Project Name: StockMaster AI

Version: 1.0

1. Project Overview

Problem Statement: Small and medium-sized milk tea shop operators in Malaysia struggle with manual inventory management. Tracking stock levels across 25+ ingredients from 22+ suppliers using paper records or spreadsheets leads to frequent stockouts, over-ordering, wasted perishable goods, and inability to compare supplier prices efficiently. Shop operators often discover critical shortages (e.g., fresh milk, boba pearls) only when items run out during peak business hours, causing lost revenue and poor customer experience.

Target Domain: AI-powered supply chain and inventory management for F&B micro-enterprises, specifically milk tea shops in Malaysia.

Proposed Solution Summary: StockMaster AI is an intelligent inventory management system that uses ILMU.ai's GLM-5.1 large language model as an agentic AI brain. The system autonomously monitors stock levels across 25 inventory items, identifies critical shortages, compares prices across 22 competing suppliers, generates optimized restocking recommendations, and drafts purchase orders (including WhatsApp messages). The AI agent understands natural language commands including Malaysian English (Manglish) expressions like 'susu tak ada liao' (milk is finished), making it accessible to local shop operators.

2. Background & Business Objective

Background of the Problem: Malaysian milk tea shops typically manage between 20–30 different ingredients spanning categories like dairy (fresh milk, oat milk, evaporated milk, non-dairy creamer), boba and toppings (tapioca pearls, popping boba, nata de coco, taro powder), tea leaves (jasmine green tea, assam black tea, matcha), sweeteners (brown sugar syrup, fructose), packaging (plastic cups, straws, sealing film, carrier bags), and janitorial supplies. Each category has multiple competing suppliers with varying prices and package sizes. Currently, most shop operators rely on memory, WhatsApp messages, or basic spreadsheets to track inventory — leading to approximately 15–20% wastage on perishable goods and frequent emergency orders at premium prices.

Importance of Solving This Issue: By automating inventory monitoring and supplier price comparison, StockMaster AI can reduce stockout incidents by an estimated 80%, lower procurement costs by 10–15% through optimal supplier selection, and free up 2–3 hours of daily manual stock-checking time. For a typical milk tea shop with RM5,000–10,000 monthly ingredient costs, this translates to RM500–1,500 in monthly savings.

Strategic Fit / Impact: This solution aligns with Malaysia's push for SME digitalization under the Malaysia Digital Economy Blueprint. By integrating Z.ai/ILMU.ai's GLM models as the reasoning engine, StockMaster AI demonstrates how advanced AI can be made accessible to micro-enterprises. The system showcases agentic AI capabilities — the LLM doesn't just answer questions but autonomously takes actions (checking inventory, comparing suppliers, placing orders) through a 25-tool API ecosystem.

3. Product Purpose

3.1. Main Goal of the System

To provide an AI-powered autonomous procurement agent that manages the complete inventory lifecycle for milk tea shops — from real-time stock monitoring and gap detection to supplier price optimization and automated order placement.

3.2. Intended Users (Target Audience)

- Milk Tea Shop Owners/Operators (Primary User)
- Shop Managers responsible for inventory and procurement
- Supply Chain Coordinators managing multiple outlet locations
- Franchise Operations Teams overseeing standardized inventory across branches

4. System Functionalities

4.1. Description

StockMaster AI operates as a dual-layer system: a React-based frontend dashboard for visual inventory monitoring, and a Python Flask backend that exposes 25+ RESTful tool endpoints. The AI agent (powered by ILMU.ai GLM-5.1) orchestrates these tools through OpenAI-compatible function calling, enabling autonomous multi-step reasoning to fulfill user requests.

4.2. Key Functionalities

Real-Time Inventory Dashboard: A web-based dashboard displaying all 25 inventory items with current stock levels, minimum thresholds, status indicators (In Stock/Low Stock/Critical), and supplier information. Supports multi-language interface (English, Bahasa Melayu, Mandarin, Tamil) and multi-currency display (RM, USD, EUR, GBP, INR, CNY, JPY).

AI-Powered Stock Gap Analysis: Automated detection of items below minimum stock thresholds with priority classification (CRITICAL for 0 stock, HIGH for below 50% of minimum, MEDIUM for below minimum). The system calculates estimated restocking costs based on current supplier prices.

Intelligent Supplier Price Comparison: Unit-price normalization engine that compares prices across suppliers with different package sizes (e.g., comparing milk sold by Case of 4 Gallons vs Case of 6 Liters). The system identifies the optimal supplier and calculates potential savings.

Automated Order Placement & Drafting: One-click order placement for critical items with automatic supplier selection based on lowest normalized price. Generates WhatsApp-ready and email-ready order messages addressed to specific suppliers.

Natural Language AI Assistant: Conversational AI interface that understands commands in English, Malay, and Manglish. Users can type queries like 'show stock gaps', 'compare prices for tapioca pearls', or 'place order for milk' and the AI agent autonomously executes the appropriate tool chain.

Supplier Debt Tracking: Maintains records of outstanding payments to each supplier, supporting debt repayment tracking and financial visibility.

Price History & Trend Monitoring: Tracks supplier price fluctuations over time (hourly, daily, monthly) with automatic background price updates every 60 seconds.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The system uses ILMU.ai's ilmu-glm-5.1 model accessed via an OpenAI-compatible API (base URL: <https://api.ilmu.ai/v1>). GLM-5.1 was selected for its

strong function-calling capabilities (supporting 25+ tool schemas simultaneously), multilingual understanding (critical for Malaysian market with Malay, English, Mandarin, Tamil users), and cost-effective inference pricing suitable for SME deployment.

4.3.2. Prompting Strategy: Multi-step agentic prompting with tool orchestration. The agent receives a comprehensive system prompt defining its role as 'StockMaster AI, an intelligent inventory management assistant for a milk tea shop in Malaysia.' The prompt includes Manglish/local expression mappings (e.g., 'habis' = finished, 'kosong' = empty/zero stock, 'tak ada liao' = out of stock). The agent autonomously decides which tools to call based on user input, executing up to 10 iterations of tool calls before providing a final response. This is a multi-step agentic prompting approach where the LLM plans and acts iteratively.

4.3.3. Context & Input Handling: The system injects full inventory data (25 items) and supplier catalog (22 suppliers with items and prices) into the LLM context window alongside user messages and conversation history. Tool results are formatted into concise summaries (capped at 2000 characters for generic results, maximum 30 items for inventory lists) to prevent context overflow. If tool results exceed limits, they are truncated with the most relevant items prioritized.

4.3.4. Fallback & Failure Behavior: When the AI receives an unrecognized command, it responds with a helpful menu of available actions ('Show stock gaps', 'Compare prices for [item]', 'Place order for [item]'). Network errors to the Tools API return structured error messages with the error type. The frontend includes a demo data fallback — if the backend is unreachable, the dashboard loads with hardcoded sample inventory data so users can still explore the interface. Additionally, the frontend has built-in pattern-matching for basic commands (gap/price/order) that works without the LLM.

5. User Stories & Use Cases

User Stories:

- As a milk tea shop owner, I want to see which ingredients are running low so that I can reorder before running out during peak hours.
- As a shop manager, I want to compare prices for the same item across different suppliers so that I always get the best deal.
- As a procurement officer, I want the AI to automatically draft WhatsApp order messages to my suppliers so that I can place orders quickly.
- As a non-technical shop operator, I want to ask the AI in Bahasa Melayu or Manglish (e.g., 'susu habis') so that I don't need to learn English commands.
- As a franchise manager, I want to view inventory valuation and category breakdowns so that I can make strategic purchasing decisions.

Use Cases (Main Interactions):

Stock Gap Detection Flow: User opens dashboard → system automatically highlights critical items (e.g., Fresh Whole Milk at 0/5 Cases) → user clicks 'Order Now' → system auto-selects cheapest supplier (Budget Dairy Supply at RM68/case vs Dairy Farms at RM73.12/case vs Premium Dairy Co. at RM85/case) → order is placed and stock is updated.

AI Natural Language Flow: User types 'susu habis, nak order' → AI recognizes 'susu' (milk) and 'habis' (finished) → calls `get_low_stock_items` tool → finds Fresh Whole Milk at 0 stock → calls `get_optimal_supplier` for milk → identifies Budget Dairy Supply as cheapest → calls `generate_order_draft` for WhatsApp → returns formatted order message ready to send.

6. Features Included (Scope Definition)

- Real-time inventory dashboard with 25 items across 9 categories (Dairy, Boba, Tea, Sweeteners, Packaging, Toppings, Janitorial, Equipment, Supplies)
- Agentic AI assistant with 25+ tool endpoints for autonomous inventory management via ILMU.ai GLM-5.1
- Multi-supplier price comparison with unit normalization engine (supports liters, kg, units, gallons, lbs)
- Automated WhatsApp and email order draft generation with supplier addressing
- Multi-language UI support (English, Bahasa Melayu, Mandarin Chinese, Tamil)
- Multi-currency display with conversion (RM, USD, EUR, GBP, INR, CNY, JPY)
- Supplier debt tracking and repayment management
- Price history tracking with hourly/daily/monthly trend views and background auto-updates
- CSV export for Excel-compatible inventory reports
- Customizable AI personality settings (Professional, Casual, Brief response styles)
- Stock gap analysis with priority classification (Critical/High/Medium)
- Bulk stock operations (update, decrease, restock multiple items at once)

7. Features Not Included (Scope Control)

- User authentication and role-based access control (no login system in current version)
- Real payment gateway integration (orders are tracked but actual payments are manual)
- Multi-branch/outlet management with separate inventory databases
- Barcode/QR code scanning for stock intake
- Mobile native application (web-only for this version, though responsive design is supported)
- Automated supplier API integration for real-time price feeds from external supplier systems
- Predictive demand forecasting based on historical sales data
- Image-based stock detection via camera/fridge photo (planned in `implementation_plan.md` but not yet implemented)

8. Assumptions & Constraints

LLM Cost Constraint: Estimated token cost per average user session is approximately 4,000–8,000 tokens (system prompt ~2,000 tokens + inventory/supplier context ~3,000 tokens + user interaction ~1,000–3,000 tokens). To manage inference costs at scale: (1) Tool results are truncated to essential information before feeding back to the LLM (max 2000 chars for generic results), (2) Inventory and supplier data are pre-formatted into concise emoji-tagged summaries rather than raw JSON, (3) Maximum 10 tool-call iterations per agent run to prevent runaway token consumption, (4) Conversation history is not persisted across sessions to avoid growing context.

Technical Constraints: The frontend is built with Create React App (React 19.2.5) and requires Node.js for development. The backend requires Python 3 with Flask >=2.3.0, flask-cors, openai SDK, and requests library. Both servers must run concurrently (React on port 3000, Flask Tools API on port 5001). Firebase Firestore is configured in `firebase.js` but currently the backend uses local JSON file storage for the hackathon demo.

Performance Constraints: GLM-5.1 API calls typically complete within 2–5 seconds per iteration. Each agent iteration adds latency, so complex multi-tool queries (up to 10 iterations) may take 10–30 seconds total. The price history generator creates synthetic fluctuation data for demo purposes ($\pm 10\%$ variation from base prices).

User Input: The system requires an ILMU.ai API key to be configured in a .env file (ZAI_API_KEY environment variable). Without a valid API key, the AI agent features revert to the frontend's built-in keyword-matching fallback (handles basic gap/price/order commands using local JavaScript logic without LLM inference).

9. Risks & Questions Throughout Development

API Dependency Risk: The entire agentic AI functionality depends on ILMU.ai's GLM API availability. If the API is down or rate-limited, the core AI features (natural language processing, multi-step tool orchestration) become unavailable. Mitigation: The React frontend includes a built-in fallback that handles basic commands (stock gaps, price comparison, order placement) using local pattern-matching without any API call.

Data Accuracy Risk: Supplier prices stored in the JSON database may become stale over time. The system generates synthetic price fluctuations ($\pm 10\%$) for demo purposes, but in a production environment, real-time supplier pricing feeds or periodic manual updates would be essential to maintain comparison accuracy.

Unit Normalization Accuracy: The price comparison engine normalizes prices across different units (gallons, liters, kg, lb, etc.) using regex-based parsing. Edge cases in unit string formats (e.g., unusual supplier descriptions) could lead to incorrect price-per-unit calculations. The current implementation handles common formats like 'Case (4 x 1 Gallon)', '3 kg bag', 'Box of 1,000' but may fail on non-standard descriptions.

Concurrent Access Risk: The JSON file-based storage (inventory.json, supplier_debt.json) does not support concurrent read/write operations. If multiple users attempt to modify inventory simultaneously, race conditions could result in data loss. Production deployment would require migration to Firebase Firestore (already configured in firebase.js with collections for suppliers, inventory, orders, and gaps).

Manglish Understanding Scope: While the AI agent's system prompt includes mappings for common Malaysian expressions (habis, kosong, tak ada liao, bawa, nak, sikit, beli, etc.), the coverage is limited to common inventory-related terms. Uncommon local expressions, regional dialects, or heavily mixed-language sentences may not be interpreted correctly by the LLM.